

Graph Connectivity as a Probe of Transformer Reasoning

A research journey, with a receipt on every claim

Barsbold Bayarerdene

August 26, 2026

Diploma research · progress report

How to read this deck

Every quantitative claim carries a **receipt** in grey underneath it — the run ID in `results/`, the source file, or the `docs/CHANGELOG.md` date where it is recorded.

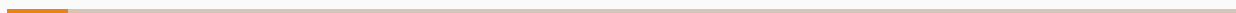
Receipts look like this: `results/20260619_222010_* · CHANGELOG 06-19`

Three slides near the end list, explicitly:

- **claims backed by a run in this repository**
- **claims backed only by a paper I read, not by my own run**
- **claims I have been repeating that I cannot back up at all**

Cross-checked against `docs/CHANGELOG.md`, `docs/yehudai-empirical.md`, and all 151 files in `results/` on 2026-08-26.

Where this started



Yehudai et al. — Depth-Width Tradeoffs for Transformers on Graph Tasks.

Given to me by my supervisor.

- a plain transformer encoder, **no message passing**
- the graph is flattened into a token sequence
- the variable is **tokenization**, **depth**, and **width**

I set out to reproduce it, without editing their source.

yehudai/run_connectivity.py · docs/yehudai-empirical.md · CHANGELOG 06-20

Reproduction: three bugs had to be worked around

blocker

cause

create_data NameError	their connectivity_adj_mat.py:207 calls a module alias that is never bound
inflated val / test	on cache hit their loader reads the train .pt for all three splits
UnpicklingError	torch $\geq$ 2.6 defaults weights_only=True and rejects their pickled PyG Data

All three are handled in the wrapper, so their source is untouched and the run is honest.

yehudai/run_connectivity.py · docs/yehudai-empirical.md §Reproduction notes

Reproduction: it reproduces

n = 50, 1 layer, width 64, 100 train graphs, 100 epochs:

tokenization	params	test acc
adj_rows	28,609	1.000
edge_list	31,873	0.926

docs/yehudai-empirical.md §2

The control that mattered

Run **my** model on **their** data. If it fails, the problem is my pipeline; if it succeeds, any later failure is the **dataset**.

model	data	test
my adj global-attn, 1 layer, 15,874 params	Yehudai n = 50	1.000 @ ep 4
their nn.TransformerEncoder	Yehudai n = 50	1.000 @ ep 20
my adj global-attn	my connectedness_hard	0.59

My pipeline beats their own implementation on their data. Everything that follows is therefore **about the data**.

docs/yehudai-empirical.md §2b · results/20260620_213659_*

Their three tokenizations

name	one token =	seq len	token dim
adj_rows	node's adjacency row $\oplus$ features	n	n + d
edge_list	onehot(u) $\oplus$ x_u onehot(v) $\oplus$ x_v	#edges	2(n+d)
lap_full	Laplacian eigenvectors $\oplus$ features	n	n + d

Honest note: I never ran lap_full **inside their pipeline**. The Laplacian result later in this deck is from my own model, on my own data.

docs/yehudai-empirical.md §1 and §4 (listed there as untested)

Comparing the GNN flavours

So I built a comparison framework

Once the reproduction worked I wanted to know whether the architecture was the variable. Implemented in one harness, one YAML per run:

- **GIN** and **rGIN** (random node features)
- **GAT**
- **GlobalAttnConv** — all-pairs attention, with a `local: true` neighbour-masked mode
- **node_edge** — Sanford-style vertex + edge + task tokens
- **BDH** (Kosowski et al. 2025)

`src/gnn.py, src/layers.py, src/graph_conv.py, src/transformer.py · configs/`

Result: the architectures land in one band

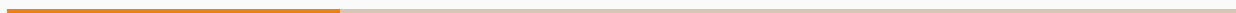
Connectivity-matrix task on `diameter_controlled`, depth 2, width 64:

layer type	params	test
global_attn	30,786	1.000
gat	14,658	0.945
bdh	202,304	0.946

`results/20260702_171141_*` (global) · `20260702_155239_*` (gat) · `20260702_163342_*` (bdh)

Same in-distribution band, wildly different parameter counts — and **all three collapse out of distribution**. That pushed my attention off architecture and onto the data.

The task was not the task



First warning: constant features collapse the model

global_attn on connectedness returned **exactly 71.00%** every epoch — no learning.

With constant features $x_i = [1]$, attention scores $Q_i \cdot K_j$ are identical for every pair, softmax is uniform, every node gets the same output, and mean-pool gives every **graph** the same embedding. The classifier predicts the majority class forever.

Replacing constant features with normalised degree: **71% → 98%**.

CHANGELOG 06-19 · results/20260619_180854_* (0.71) vs 20260619_181808_* (0.98)

Second warning: that 98% was not reasoning either

connectedness builds Erdős–Rényi graphs near the threshold $p \approx \log(n)/n$, where disconnection is **almost always caused by isolated vertices**.

Measured directly, seed 42, 1000 graphs:

trivial rule	accuracy
majority class (connected)	0.679
connected $\iff$ min degree ≥ 1	0.982

94.4% of disconnected graphs have a degree-0 node; **0%** of connected ones do. The node feature **was** degree — so the model only had to find a zero.

CHANGELOG 06-19 · memory connectedness-dataset-leak · reproducible from dataset.py

Third warning: Laplacian features leak connectivity outright

The Laplacian's zero-eigenvalue eigenvectors are **localised on connected components**, so component membership becomes linearly separable in the input.

Same model, same data, same depth — only the encoding changes:

dataset	depth × width	no LPE	with LPE
connectedness	1 × 64	0.91	1.00
connectedness_hard	1 × 64	0.545	1.00

results/20260619_221812_* (lpe 0 → 0.545) vs 20260619_222010_* (lpe 16 → 1.00)

The Laplacian leak survives extreme compression

If it were reasoning, shrinking the model should break it. It does not:

width	lpe dim	test on connectedness_hard_adj
64	16	1.00
24	16	1.00
16	8	1.00
8	4	1.00

A width-8 model with 4 Laplacian dimensions still scores 1.00. The answer is **in the input**, not in the computation.

results/20260621_010109_*,_010500_*,_010857_*,_011108_* · CHANGELOG 06-21

connectedness_hard



So I built a dataset that removes the shortcuts

`make_connectedness_hard` — both classes are two dense, internally-connected blobs:

- every node has degree $\geq 2 \rightarrow$ **no isolated-vertex shortcut**
- connected vs disconnected differs by **exactly one bridge edge**
- edge counts matched across classes $\rightarrow$ **no edge-count shortcut**
- verified: min-degree and mean-degree rules both back at chance

Later `connectedness_hard_diam` added a diameter axis (sparse cycle blobs + 0–3 chords, diameters 3–13, degree- and edge-count-matched per class).

`dataset.py` · CHANGELOG 06-19 and 07-03 · docs/DATASETS.md

And then nothing could learn it

Every run below: connectedness_hard, binary label, lpe_dim: 0, full data.

depth	width	features	test
1	2	degree	0.500
1	64	degree	0.545
1	128	degree	0.535
3	64	degree	0.520
4	64	degree	0.520 – 0.545
5	32	degree	0.530
1	20 – 128	adj_rows	0.565 – 0.640
4	128	adj_rows	0.520
8	32	adj_rows	0.585 – 0.600

results/20260619_221642_*, _221812_*, _224205_*, _224932_*, _234958_*, _235233_*, 20260620_104231_*, _172317_*, 20260623_213906_* ... _234038_*

Depth 1 to 8, width 2 to 128 — a flat band at chance

Roughly **20 runs**, depths 1–8, widths 2–128, two encodings, up to 500 epochs. Every one lands between **0.50 and 0.64**.

The best result in the whole grid — 0.64, one layer, width 32, adjacency rows — is barely above the 0.50 chance line and does not improve with depth or width.

Grid assembled from `results/` on 2026-08-26; individual IDs on the previous slide. Not a pre-registered sweep — it accumulated across sessions 06-19 → 06-23.

It is generalization that fails, not capacity

The control that makes the negative result an argument. Same model, 4×64 , `lpe_dim: 0` — only the number of training graphs changes:

train set	test	reading
10 graphs	1.00	the model can fit this data
100 graphs	0.76	degrading
full data	0.545	chance

Capacity is not the blocker. A sharp cliff between memorising 10 graphs and failing on 100+ is the signature of **no transferable circuit**.

`results/20260620_170729_overfit10_* · _171029_overfit100_* · _172317_* (500 ep)`

A mechanism for one of the failures

The Sanford-style node_edge transformer sat at train loss **exactly** $0.693 = \ln 2$ for entire runs. Raising lr $0.0005 \rightarrow 0.005$ changed nothing.

Cause: with `node_id_mode: learned`, node identity comes from a shared embedding indexed by within-graph position. The blob split varies per graph, so position 5 is in blob A for some graphs and blob B for others — the same embedding row is pushed in opposite directions, the gradients cancel, and the optimiser parks at max entropy.

$\text{lr} \times 0 = 0$ — a bigger step cannot escape a **structurally** vanishing gradient.

CHANGELOG 06-20 · `reports/tokenization.typ`. **No results JSON** — this run predates the run logger; the evidence is the changelog entry and the writeup.

But connectedness_hard is not clean either

This undercuts the previous slides, so I have to state it:

model	features	train	test
3-layer GIN	degree	0.923	0.975
3-layer GIN	random (rGIN)	0.970	0.630

0.975 at **depth 3, on graphs of diameter 4–8** cannot be global reasoning. It is **local bridge-detection** — a shortcut my construction failed to kill.

results/20260624_183646_* · _182818_* · CHANGELOG 06-25

Sanford et al.



Sanford et al. 2024, Understanding Transformer Reasoning Capabilities via Graph Algorithms (NeurIPS 2024).

- graph $\rightarrow$ vertex tokens + edge tokens + a task token; answer read from the task token
- depth $\leftrightarrow$ parallel-computation rounds: 1 layer $\approx$ 1 MPC round
- tasks classed as **retrieval** (1 layer), **parallelizable** (connectivity, $\log n$ depth), and **search / DP** (shortest path, hardest)

So connectivity should be reachable at depth $\log n$ — and my depth-8 runs on graphs of $n \approx 12\text{--}24$ sit above that threshold.

docs/CONFIG.md §node_edge · memory thesis-sanford-graph-reasoning · reports/message-passing.typ

Two things I have been saying that I cannot back up

1. “They need $m = O(N^\epsilon)$ width.”

Plausible — it is the standard MPC-simulation regime — but this parameter appears **nowhere** in my notes. I am recalling it from the paper, not from anything I wrote down.

2. “They used an LLM for the graph task execution.”

Appears **nowhere** in the repository — I have reading companions for Yehudai, Ye, Kosowski and Airale, **not for Sanford**. Probably imprecise too: they appear to do both LLM prompting and from-scratch training.

Action: write `reports/sanford-2024-notes.typ` — the largest hole in my coverage.

Ye et al.



Ye, Fu, Jia, Sharan — Transformers Provably Learn Algorithmic Solutions for Graph Connectivity, But Only with the Right Data (ICML 2026).

- two channels: an **algorithmic** channel and a **heuristic** channel
- a depth- L model has reachability **capacity** 3^L
- the lever: train only on graphs **within capacity** (diameter $\leq 3^L$), so the heuristic channel is starved and the algorithmic one is forced

I implemented both their target and their lever.

reports/ye-2026-notes.typ (full reading companion) · ye_connectivity.py · src/connectivity.py · configs/connectivity_hard.yaml

The adversarial probe: their model is a density detector

I built two out-of-distribution probes: **two disconnected cliques**, and **the same two cliques joined by one bridge**. Exact-match on the reachability matrix:

regime	train EM	ER test	2 cliques, disconnected	+ bridge
raw	1.000	0.578	0.000	1.000
within	1.000	0.698	0.000	1.000
within (small)	0.453	0.380	0.000	1.000

0.0 on disconnected, 1.0 on connected is the exact signature of a model that always answers “everything is reachable”. It is reading **density**, not connectivity.

results/20260625_002255_ye_connectivity_raw.json · _001934_ye_connectivity_within.json · _000927_* · probes in ye_connectivity.py

And their lever does not fix it

The decisive column is the last one: **within-capacity training changes nothing about the clique probe.** Both regimes fail identically.

Their data condition suppresses the heuristic **on their distribution**. On an adversarial probe, the heuristic is all that is left.

This is the sharpest single piece of evidence in the whole project, and it took two runs.

Same three JSONs. Config on both: depth 2, $n = 60$, $p = 0.05$, 3000 train, 300 epochs, capacity 9.

Why the lever could not engage — the catch-22

Diagnostics (ρ_{hard} = fraction of reachable pairs beyond capacity, plus diameter histograms) show **why**:

- **ER graphs**: diameters concentrate tightly. At capacity 9 everything is within capacity (the filter removes nothing) or, at lower capacity, everything is beyond it (the filter removes all training data). **No regime gives both.**
- **diameter-controlled caterpillars**: the filter engages (ρ_{hard} 0.04 raw vs 0.00 within) — but caterpillars are degree-uniform, so **the degree heuristic never forms**, and there is nothing to suppress.

CHANGELOG 06-25 §“The Ye data lever is a catch-22 on our setups”

A second confound: global attention voids the capacity wall

Their 3^L capacity assumes **local** mixing — a node combines with its neighbours.

My GlobalAttnConv is **global all-pairs**: one layer reaches any node. It solves even diameter-18 graphs at capacity 9, so the lever cannot bind at all.

Adding `local: true` (attention masked to graph neighbours + self, 1 hop per layer) made the capacity wall appear: `test < test_within` once graphs exceed reach.

Local attention = GAT = a GNN. **More transformer-like (global) → more shortcutting; more GNN-like (local) → more genuine computation.**

CHANGELOG 06-25 · `local` flag in `src/layers.py`

Today's runs — provisional

14 runs today re-testing the lever with the clique probes wired into the main harness. The picture is **not** the same as June, and I have not written it up yet:

regime	depth	test	clique disc	+ bridge
raw	2	0.955	1.00	0.00
raw	6	0.993	1.00	1.00
within	2	0.975	1.00	1.00
within	2	0.930	1.00	0.00

The failure has **inverted** — these models now get the disconnected cliques right and the bridged ones wrong, and the result is unstable across otherwise-identical runs.

results/20260826_18* (14 runs) · **no CHANGELOG entry for 08-26 yet; dataset kwargs are not recorded in these configs**

What did work

Supervision density, not architecture

The June headline. **Same graphs**, two different targets:

target	result	supervision
binary “is it connected?”	stalls at $\ln 2$	1 bit / graph
$n \times n$ reachability matrix R	0.97 exact-match	n^2 bits / graph

Producing R **requires** computing reachability for every pair; once you have R , the binary answer is `all(R == 1)` — free. So the hard part was never the decision. It was that the binary loss gave gradient descent nothing to grip.

CHANGELOG 06-25 §“Supervision density is the blocker, not the architecture” · `configs/connectivity_hard.yaml`

Traces: the same lesson, arranged in time

If dense supervision in **space** works, does dense supervision in **time**? A depth-2 decoder emits a full BFS trace, then reads the answer off it:

dataset	with trace	answer-only	by diameter
diameter_controlled	0.964	0.510	flat 0.92–0.99, diam 2–18
connectedness_hard_diam	0.998	0.511	flat, diam 3–13
connectedness_hard	0.997	—	flat 0.985–1.00, diam 2–8

Identical architecture, data and optimiser; the trace is the only difference. The **flatness in diameter** is the point — the trace converts diameter into sequence length.

results/20260705_005620_*, 20260705_155408_* (control), 20260708_153126_*, 20260708_180117_*

The ledger



claim

Yehudai's connectivity experiment reproduces

ER connectedness is 98.2% solvable by “min degree ≥ 1 ”

Laplacian features take `connectedness_hard` 0.545 $\rightarrow$ 1.00

...and still score 1.00 at width 8

`connectedness_hard` sits at 0.50–0.64 across depth 1–8, width 2–128

the failure is generalization, not capacity

architectures land in one band

Ye-style models score 0.0 / 1.0 on the clique probe, in **both** regimes

dense targets (matrix, trace) learn what the binary target cannot

receipt

`docs/yehudai-empirical.md`

CHANGELOG 06-19

20260619_221812 vs _222010

20260621_011108

20 runs, 06-19 $\rightarrow$ 06-23

`overfit10` $\rightarrow$ 1.00

20260702_1711/1552/1633

three `ye_connectivity` JSONs

CHANGELOG 06-25, 07-05 $\rightarrow$ 07-08

These are fine to state — but they must be cited to the paper, not presented as my findings.

- Sanford's depth $\leftrightarrow$ MPC-rounds correspondence, and connectivity needing $\log n$ depth
- Sanford's $m = O(N^\epsilon)$ width condition — **I have no note of this at all**
- Ye's 3^L capacity bound and their theorem's hypotheses
- Yehudai's `lap_full` tokenization behaviour — **I never ran it in their pipeline**
- Merrill & Sabharwal's result that CoT strictly extends fixed-depth power

Claims I cannot back up — and one I had backwards

1. “Sanford used an LLM for the graph task execution.”

Nothing in the repository. No Sanford reading companion exists. Probably imprecise — they appear to do both LLM prompting and from-scratch training. **Re-read before repeating.**

2. “connectedness_hard cannot be learned.”

Overstated. Four things learned it: LPE (1.00 — a leak), 3-layer GIN with degree features (0.975 — a shortcut), the reachability-matrix target (0.97 — genuine), and BFS traces (0.997 — genuine). The correct claim is narrower: **the binary label is not learnable by these encoders at any depth or width I ran.**

3. “Ye’s method didn’t work on connectedness_hard.”

I had this backwards. Their **lever** would not engage on my distributions; their **matrix target** reached 0.97 on exactly the graphs where the binary task stalls.

What is missing before any of this is defensible

1. **Write `reports/sanford-2024-notes.typ`**. Two of my standing claims depend on a paper I have no notes on.
2. **Kill the local bridge-detection shortcut in `connectedness_hard`** (GIN-degree 0.975 at depth < diameter). Every negative result rests on this dataset.
3. **Re-run the Ye lever on `connectedness_hard_diam`** with `local: true` — it was built in June precisely to break the catch-22, and was never used for it.
4. **Document today's 14 runs** and explain why the clique-probe failure inverted.
5. **Re-run the depth $\times$ width grid as a pre-registered sweep** — the current one accumulated across four sessions with drifting configs.

Every paper I have read makes a claim about what a transformer **can represent**, or about **a dataset**. What none of them establishes — and what my runs keep hitting — is whether gradient descent ever **finds** the construction on data with no statistical shortcut.

Candidate questions, with evidence status and decisive experiments, are written up in `docs/RESEARCH-QUESTIONS.md`.